bias_initialization.py — technical

`src/moe_pruning/pruning/bias_initialization.py · 138 lines · moe_pruning.pruning.bias_initialization`

Purpose

Closed-form bias initialisation and residual diagnostics from running sums.

Formulas

```
w_t = s_t * g_t^2; b = sum_t w_t E_t / sum_t w_t

sum_t w_t ||E_t - b||^2 = sum_t w_t ||E_t||^2 - 2 b . sum_t w_t E_t + ||b||^2 sum_t w_t

weighted_rel_residual = that / sum w||E||^2; gate_weighted uses w = g^2; per_token = g^2-residual / layer tokens
```

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class UnobservedExpertError`

lines 42-43 · bases ValueError

Raised when an expert never seen in calibration would need a bias.

`weighted_mean`

```
def weighted_mean(weighted_sum: torch.Tensor, weight_total: float | torch.Tensor) -> torch.Tensor | None
```

lines 46-51

sum_t w_t x_t / sum_t w_t, or None when the denominator is (numerically) zero.

1. Return `None` when the weight total is not > 1e-12, else `weighted_sum / total`.

`weighted_residual`

```
def weighted_residual(sq_sum: float, weighted_sum: torch.Tensor, weight_total: float, b: torch.Tensor) -> float
```

lines 54-58

sum_t w_t ||x_t - b||^2 from running sums (clamped at 0 for rounding).

`class BiasInitResult`

lines 62-77 · dataclass

Method used, fallback flag, token count, bias norm and residual diagnostics.

Field	Type	Default
`layer_id`	str	
`expert_id`	int	
`method`	str	
`fallback`	bool	
`token_count`	int	
`weight_sum`	float	
`bias_norm`	float	
`weighted_relative_residual`	float | None	
`gate_weighted_relative_residual`	float | None	
`layer_output_sq_error_per_token`	float | None	

`BiasInitResult.to_dict`

```
def to_dict(self) -> dict
```

lines 76-77

Convert to a plain dictionary for JSON reports.

`initialize_bias`

```
def initialize_bias(stats: LayerStatistics, expert_id: int, allow_unobserved: bool=False) ->
tuple[torch.Tensor, BiasInitResult]
```

lines 80-138 · raises UnobservedExpertError

Compute b_e (float64) for one expert from accumulated statistics.

1. Try the s*g^2-weighted mean; if None: gate-weighted mean (g^2); then plain mean over all calibration tokens; then (only if unobserved experts are allowed) zeros - else `UnobservedExpertError`. Any fallback logs a warning.
2. Compute the residual diagnostics with `weighted_residual` (clamped at 0 for rounding).
3. Return the float64 bias and a `BiasInitResult` (method, fallback flag, training-token count, weight sum, norm, residuals).

Module constants

Name	Value
`EPS`	1e-12

Errors raised

Function	Exception	Message (f-string fields in braces)
`initialize_bias`	UnobservedExpertError	{stats.layer_id} expert {e} was never activated on the calibration data; refusing to invent a bias (PRUNE_UNOBSERVED_POLICY=exclude).